

NAME

nhedit - edit or view an image header interactively or using a

USAGE

```
nhedit images fields value comment
```

PARAMETERS**images**

Template specifying the images to be edited.

fields

Template specifying the fields to be edited in each image. The template is expanded independently for each image against the set of all fields in the image header. Special values for fields includes 'default_pars' that works only with a command file; 'add_blank' to add a blank field value with a value; 'add_textf' to add a text file content to the header. See description for more details.

value

Either a string constant or a general expression (if the first character is a left parenthesis) to be evaluated to compute the new value of each field. With the rename switch the value is the new field name (keyword). A single expression is used for all fields. The special value "." causes the value of each field to be printed rather than edited.

comment

String constant for the comment section of the header card. This value will replace the existing comment of a header or clear it if is empty (""). The special value "." causes the field to be printed rather than edited.

comfile = ""

Alternate command file. If specified, the *fields*, *value*, and *comment* parameters are ignored and commands are taken from the named file. See below for a detailed discussion and examples.

after = ""

Insert the new field after the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

before = ""

Insert the new field before the named "pivot keyword". If this keyword does not exist in the header, the new keyword is added to the end of the image header.

add= no

Change the operation of the editor from update to add new field. If field already exists it is edited. If this option is selected the field list may name only a single field. The add switch takes precedence over the addonly, delete, and rename switches.

addonly= no

Change the operation of the editor from update to add a field already exists it is not changed. If this option is selected the field list may name only a single field. The addonly switch takes precedence over the delete and rename switches.

delete = no

Change the operation of the editor from update to delete field. The listed fields are deleted from each image. This takes precedence over the rename switch.

rename= no

Change the operation of the editor from update field to rename field. The listed fields are renamed in each image if they exist. The value parameter specifies the new keyword name. There is no error if the field does not exist. The comment value is ignored since this operation only affects the field name.

verify = yes

Interactively verify all operations which modify the image database. The editor will describe the operation to be performed, prompting with the new value of the parameter in the case of a return or "yes" to complete the operation, or enter a new value explicitly as a string. Respond with "no" if you do not wish to change the value of the parameter.

show = yes

Print a record of each operation which modifies the database upon the standard output. Old values are given as well as new values, making it possible to undo an edit operation.

update= yes

Enable updating of the image database. If updating is disabled the edit operations are performed in memory but image headers will not be updated on disk.

DESCRIPTION**1. Basic Usage**

The most basic functions of the image header editor are modification and inspection of the fields of an image header. Both the "standard" and "user" fields may be edited in the same fashion, although not all standard fields are writable. For example, to change the value of the standard field "title" of the image "m74" to "sky flat" and enter a comment field we would enter the following command.

```
cl> nhedit m74 title "sky flat" "comment field"
```

If *verify* mode is selected the editor will print the old value of the field and query with the new value, allowing some other value to be entered instead, e.g.:

```
cl> nhedit m74 title "sky flat" "comment field"
m74,i_title ("old title" -> "sky flat"):
```

To accept the new value shown to the right of the arrow, type carriage return or "yes" or "y" followed by carriage return. To continue without changing the value of the field in question enter "no" or "n" followed by carriage return. To enter some other value merely type in the new value. If the new value is one of the reserved strings, e.g., "yes" or "no", enter it preceded by a backslash. If verification is enabled you will also be asked if you want to update the header, once all header fields have been edited. This is your last chance to change your mind before the header is modified on disk. If you respond negatively the image header will not be updated, and editing will continue with the next image. If the response is "q" the editor will exit entirely.

To conveniently print the value of the field "title" without modifying the image header, we repeat the command with the special value "." and "." for the comment portion.

```
cl> nhedit m74 title . .
```

To print (or edit) the values of all header fields a given.

```
cl> nhedit m74 * . .
```

To print (or edit) the values of only a few fields the field template may be given as a list.

```
cl> nhedit m74 w0,wpc . .
```

To print the value of one or more fields in a may be given. Both image templates and field templates may be given if desired.

```
cl> nhedit n1.* exp . .
```

Abbreviations are not permitted for field names, i.e., the given template must match the full field name. Currently, field name matches are case insensitive since image headers are often converted to and from FITS headers, which are case insensitive.

2. Advanced Usage

The header editor is capable of performing global edits on entire image databases wherein the new value of each field is computed automatically at edit time and may depend on the values of other fields in the image header. Editing may be performed in either batch or interactive mode. An audit trail may be maintained (via the *show* switch and i/o redirection), permitting restoration of the database in the event of an error. Trial runs may be made with updating disabled, before committing to an actual edit which modifies the database.

The major editing functions of the *nhedit* task are the following:

update	modify the value of a field or fields
addonly	add a new field
add	add a new field or modify an old one
delete	delete a set of fields
rename	rename a set of fields

In addition, *nhedit* may be used merely to inspect the values of the header fields, without modification of the image database.

2.1 Special header fields

add_blank	Add blank keyword field with optional comment
ex: nhedit add_blank " this is a comment with no kw"	
add_textf	Add the content of a text file into the header
ex: nhedit add_textf "my_text.txt" add+	

All keyword addition can be inserted after or before an existent keyword; use the 'after' and 'before' parameter.

2.2 Input commands from a command file.

All header editing command can be put together in a

```
nhedit file*.fits comfile=command_file.txt
```

2.3 Standard header fields

The header editor may be used to access both the standard image header fields and any user or application defined fields. The standard header fields currently defined are shown below. There is no guarantee that the names and/or usage of these fields will not change in the future.

i_ctime	int	create time
i_history	string	history comments

i_litime	int	time when min,max last updated
i_maxpixval	real	maximum pixel value
i_minpixval	real	minimum pixel value
i_mtime	int	time of last modify
i_naxis	int	number of axes (dimensionality)
i_naxis[1-7]	int	length of each axis
i_pixfile	string	pathname of pixel storage file
i_pixtype	int	pixel datatype code
i_title	string	title string

The standard header field names have an "i_" prefix to reduce the possibility of a name collision with a user field name, and to distinguish the two classes of parameters in templates. The prefix may be omitted provided the simple name is unique.

2.4 Field name template

The form of the field name list or template parameter *fields* is equivalent to that of a filename template except that "@listfile" is not supported, and of course the template is expanded upon the field name list of an image, rather than upon a directory. Abbreviations are not permitted in field names and case is not significant. Case is ignored in this context due to the present internal storage format for the user parameters (FITS), which also limits the length of a user field name to 8

2.5 Value expression

The *value* parameter is a string type parameter. If the first character in the string is a left parenthesis the string is interpreted as an algebraic expression wherein the operands may be constants, image header variables (field names), special variables (defined below), or calls to intrinsic functions. The expression syntax is equivalent to that used in the CL and SPP languages. If the value string is not parenthesized it assumed to be a string constant. The *value* string will often contain blanks, quotes, parenthesis, etc., and hence must usually be quoted to avoid interpretation by the CL rather than by the header editor.

For example, the command

```
cl> nhedit m74 title "title // ';'ss'" "."
```

would change the title to the literal string constant "title // " whereas the command

```
cl> nhedit m74 title "(title // ';'ss')"
```

would concatenate the string ";ss" to the old title string. We require parenthesis for expression evaluation to avoid the need to doubly quote simple string constant values, which would be even more confusing for the user than using parenthesis. For example, if expressions did not have to be parenthesized, the first example in the basic usage section would have to be entered as shown below.

```
cl> nhedit m74 title '"sky flat"' # invalid command
```

Expression evaluation for *nhedit*, *hselect*, and similar tasks is carried out internally by the FMTIO library routine **evexpr**. For completeness minimal documentation is given here, but the documentation for *evexpr* itself should be consulted if additional detail is required or if problems occur.

2.5.1 operators

The following operators are recognized in value expressions. With the exception of the operators "?", "=?", and "@", the operator set is equivalent to that available in the CL and SPP languages.

+	-	*	/	arithmetic operators
**				exponentiation
//				string concatenation
!	-			boolean not, unary negation
<	<=	>	>=	order comparison (works for strings)
==	!=	&&		equals, not equals, and, or

?	=	string equals pattern
?	:	conditional expression
@		reference a variable

The operators "=", "&&", and "||" may be abbreviated as "=", "&", and "|" if desired. The ?= operator performs pattern matching upon strings. For example, the boolean expression shown below will be true whenever the field "title" contains the substring "sky".

```
(title ?= '*sky*')
```

The conditional expression operator '??', which is patterned after a operator in C, is used to make IF ELSE like decisions within an expression. The syntax is as follows:

```
<bool_expr> '??' <true_expr> ':' <>false_expr>
```

e.g., the expression

```
((a > b) ? 1 : 0)
```

has the value 1 if A is greater than B, and 0 otherwise. The datatypes of the true and false expressions need not be the same, unlike a compiled language. Note that if the parenthesis are omitted ambiguous forms of the expression are possible, e.g.:

```
(a > b) ? 1 : a + 1
```

could be interpreted either as

```
((a > b) ? 1 : a) + 1 or as (a > b) ? 1 : (a + 1)
```

If the parenthesis are omitted the latter interpretation is assumed.

The operator @ must be used to dereference variables that have names with funny (nonalphanumeric) characters in them, forcing the variable name to be given as a string constant. For example, the value of the expression

```
@ "co-flag"
```

is the value of the variable "co-flag". If the variable were referenced directly by name the "-" would be interpreted as the subtraction operator, causing an unknown variable reference (e.g., to "co"). The operand following the @ may be any string valued expression. The @ operator is right associative, hence the construct "@@param" is the value of the parameter named by the value of the parameter "param".

An expression may contain operands of datatypes bool, int, real, and string. Mixed mode expressions are permitted with automatic type coercion. Most type coercions from boolean or string to other datatypes are illegal. The boolean constants "yes" and "no" are predefined and may be used within expressions.

2.5.2 intrinsic functions

A number of standard intrinsic functions are recognized within expressions. The set of functions currently supported is shown below.

abs	acos	asin	atan	atan2	bool	cos
exp	int	log	log10	max	min	mod
nint	real	sin	sqrt	str	tan	

The trigonometric functions operate in units of degrees rather than radians. The *min* and *max* functions may have any number of arguments up to a maximum of sixteen or so (configurable). The arguments need not all be of the same datatype.

A function call may take either of the following forms:

```

    <identifier> '(' arglist ')'
or
    <string_expr> '(' arglist ')'

```

The first form is the conventional form found in all programming languages. The second permits the generation of function names by string valued expressions and might be useful on rare occasions.

2.5.3 special operands

As noted earlier, expression operands may be constants, variables (header fields), function calls, or references to any of the special variables. The following special variables are recognized within expressions:

.	A string constant, used to flag printing
\$	The value of the "current field"
\$F	The name of the "current field"
\$I	The name of the "current image"
\$T	The current clock time (an integer value)

These builtin variables are especially useful for constructing context dependent expressions. For example, the value of a field may be incremented by 100 by assigning it the value "\$ + 100".

EXAMPLES

1. Globally edit the database "n1", setting the value of the string parameter "obs" to "sky" if "s-flag" is 1, to "obj" otherwise.

```
cl> nhedit n1.* obs '(@"s-flag" == 1 ? "sky" : "obj")' "Observation value"
```

2. Globally edit the same database, replacing the value of the parameter "variance" by the square root of the original value.

```
cl> nhedit n1.* var '(sqrt(var))' "Variance value"
```

3. Replace the values of the fields A and B by the absolute value of the original value:

```
cl> nhedit n1.* a,b '(abs($))' 'Absolute value'
```

4. Add a blank field with a comment after a

```
cl> nhedit file.fits add_blank "INSTRUMENT DESCRIPTION " after=k5dx add+
```

Notice the use of the special field value 'add_blank' which will be replaced by a blank keyword in the header.

5. Add HISTORY card before a given keyword

```
cl> nhedit file.fits history \
    "History text from column 9 to 80, no quotes" before=wcsdim add+
```

6. Run a command file through the first 50 extensions

```
cl> for(i=1;i<51;i=i+1) {
    nhedit("mymef["//i//"]",comfile="home$hh.in")
}
```

7. Add a text file to the header. This will be put as HISTORY lines with text appropriately split when long lines are encountered. Start putting the text after the keyword KEYWN.

```
cl> nhedit add_textf "mytext_file.tx" after=KEYWN add+
```

8. Run nhedit through all the extensions in a

```
cl> for(i=1;i<7;i=i+1)
      nhedit("mymef.fits["//i//"]",comfi="home$myheader.txt")
```

9. Run several fits files with the same set of header commands from the file "hdrc.txt".

```
cl> nhedit file*.fits commfile=hdrc.txt
```

As an example the 'hdrc.txt' content can be: (Notice the 'default_pars' command)

```
#
# Sample command file for nhedit task.
#
# Establish the default parameters for the rest of the commands.

default_pars upda+ add+ show- veri-

# Notice the use of commas if you desire.
"DETECTOR" 'Newfirm', "comment string"
ONELE 'A' "comment to A"
#
# Now delete a keyword
ONELE1 del+ show+
add_blank " /blank keyw"

# add a boolean value T
ONELE1 '(1==1)', "comment to A"

"DETSIZE", '[1:2048,1:2048]'
"ENVTEM", 1.56000000000000E+01

# Add a field with string value 'T'
ONELEi2 'T'

bafkeyw1 123.456 "comment to key1" before="WCSDIM" addonly+ show-
add_blank "COMMENT FOR A BLANK" after="FR-SCALE" add+ show-
history "this is a hist to append" add+ show-
history "this is a hist 22 after trim pkey" after="TRIM" add+ show-
comment "this is a comment" after="FR-SCALE" add+ show-
# END OF HDRC.TXT FILE
```

SEE ALSO

hselect, hedit, mkheader, imgets, imheader